

01 S3 to Lambda Read CSV with Pandas

Lab 1: S3 to Lambda Read CSV with Pandas

This lab demonstrates how to trigger a Lambda function when a file is uploaded to an S3 bucket, read the CSV content of that file using the Pandas library within the Lambda, and print its content.

1. Objective

Read CSV file content from an S3 bucket using an AWS Lambda function, triggered by an S3 upload event, and leverage the Pandas library for data processing.

2. Prerequisites

- An AWS account with console access.
- An S3 bucket (e.g., `sample-data00073`).
- A sample CSV file (e.g., `storedata.csv`) with a few records.
- Estimated Time: 20 minutes

3. Detailed Steps

Step 1: Create a Lambda Function

1. Log in to the **AWS Console**.
2. Navigate to **Lambda**.
3. Click **Create function**.
4. Choose **Author from scratch**.
5. Name the function: `S3toLambdaRead` .
6. Runtime: **Python 3.10** (or latest available).
7. Click **Create function**.
8. In the **Configuration** tab of the new Lambda function, go to **General configuration** and edit the **Timeout** to at least **1 minute** (e.g., 1:00). Click **Save**.

Step 2: Add a Lambda Layer for Pandas

1. In the Lambda function's overview, under the **Layers** section, click **Add a layer**.
2. Choose **AWS layers**.
3. Select **AWSSDKPandas-Python310** (or the version matching your Python runtime).
4. Choose the latest version available.
5. Click **Add**.

Alternative Layer Option If the AWS-provided pandas layer is not available in your region, you can search for "pandas lambda layer" + your region in AWS documentation or use a third-party ARN.

Step 3: Configure S3 Trigger

1. In the Lambda function's overview, click **Add trigger**.

2. Select **S3** as the source.
3. Choose your S3 bucket (e.g., `sample-data00073`).
4. Event types: Select **All object create events** or specifically **s3:ObjectCreated:Put**.
5. (Optional) Configure prefix and suffix if needed; otherwise, leave empty.
6. Acknowledge the warning about existing overlapping suffixes/rules if any.
7. Click **Add**.

Overlapping Suffixes If you encounter an error like "Cannot have overlapping suffixes", it means another Lambda function is already configured with a similar trigger on the same S3 bucket and event type. You may need to delete the previous conflicting trigger or Lambda function.

Step 4: Update Lambda Execution Role Permissions

1. In the Lambda function's **Configuration** tab, go to **Permissions**.
2. Click on the **Role name** (e.g., `S3toLambdaRead-role- ...`). This will open the IAM console.
3. In the IAM role's permissions, click **Add permissions** -> **Attach policies**.
4. Search for and attach the `AmazonS3ReadOnlyAccess` policy (or create a more granular policy to read from your specific bucket).
5. Click **Add permissions**.

Step 5: Write Lambda Code

In the Lambda function's **Code** tab, replace the default `lambda_function.py` code with the following Python script:

4. Code Snippets (Lab 1)

```
import boto3
import pandas as pd
from io import StringIO
import json

def lambda_handler(event, context):
    print("Event received:", json.dumps(event, indent=2))

    try:
        # Parse bucket name and key from the S3 event
        # The event contains a list of records
        record = event['Records'][0] # Get the first record
        bucket_name = record['s3']['bucket']['name']
        s3_key = record['s3']['object']['key']

        print(f"Bucket Name: {bucket_name}")
        print(f"S3 Key: {s3_key}")

        # Create an S3 client for programmatic access
        s3_client = boto3.client('s3')
```

```

# Get the object content from S3
response = s3_client.get_object(Bucket=bucket_name, Key=s3_key)
body = response['Body'].read().decode('utf-8')

# Read the content into a Pandas DataFrame
df = pd.read_csv(StringIO(body))

# Print the DataFrame content
print("DataFrame shape:", df.shape)
print("DataFrame content:")
print(df.to_string())

# Print basic statistics
print("\nDataFrame info:")
print(df.info())

return {
    'statusCode': 200,
    'body': json.dumps({
        'message': 'Successfully processed S3 file.',
        'bucket': bucket_name,
        'key': s3_key,
        'rows': len(df),
        'columns': len(df.columns)
    })
}

except Exception as e:
    print(f"Error processing file: {str(e)}")
    return {
        'statusCode': 500,
        'body': json.dumps({
            'message': 'Error processing S3 file.',
            'error': str(e)
        })
    }
}

```

- Click **Deploy** after pasting the code.

5. Verification

Step 1: Upload a File to S3

1. Navigate to your S3 bucket (e.g., sample-data00073).
2. Click **Upload**, then **Add files**.
3. Select your storedata.csv file and click **Upload**.

Step 2: Check Lambda Logs

1. Go back to your Lambda function (S3toLambdaRead).
2. Navigate to the **Monitor** tab.
3. Click **View CloudWatch logs**.

4. Click on the latest **Log stream**.
5. You should see logs indicating:
 - The S3 event trigger JSON.
 - The extracted bucket name and key.
 - The content of your CSV file printed as a Pandas DataFrame.
 - DataFrame statistics and information.

Step 3: Verify Lambda Metrics

1. In the **Monitor** tab, check the **Invocations**, **Duration**, and **Error count** metrics.
2. Ensure there are no errors and the function executed successfully.

6. Cleanup

Step 1: Delete Lambda Function

1. In the Lambda console, select `S3toLambdaRead` and click **Actions** -> **Delete function**.
2. Type "delete" to confirm.

Step 2: Clean S3 Event Notifications

1. In your S3 bucket, go to **Properties** -> **Event notifications**.
2. Select the event notification created for the Lambda and click **Delete**.

Step 3: Delete S3 Bucket (Optional)

If this bucket was created specifically for the lab, you can delete it after ensuring it's empty.

7. Troubleshooting

Common Issues:

1. **Permission Errors:** Ensure the Lambda execution role has S3 read permissions.
2. **Layer Not Found:** Verify the pandas layer is compatible with your Python runtime version.
3. **Timeout Errors:** Increase the Lambda timeout for large CSV files.
4. **Import Errors:** Ensure the pandas layer is properly attached and deployed.

Testing Tips:

1. Start with a small CSV file (< 1MB) for initial testing.
2. Check CloudWatch logs immediately after uploading to S3.
3. Use the Lambda test feature with a sample S3 event for debugging.